

✓ REPASO - SIMULACRO PARCIAL 1 - 5K1 y 5k2

✓ Ejercicio 1

Una empresa de telecomunicaciones está tratando de identificar si un cliente es propenso a abandonar el servicio (churn) o no. La empresa ha recopilado datos sobre los comportamientos de los clientes, como el número de quejas recibidas y el porcentaje de uso mensual del servicio.

La meta es clasificar a los clientes en dos categorías:

1. **Churn (Abandono):** Si el cliente es propenso a abandonar el servicio (Verdadero - Clase 1).
2. **No churn (No Abandono):** Si el cliente es fiel al servicio (Falso - Clase 0).

La empresa proporciona un pequeño conjunto de datos de ejemplo para desarrollar un modelo predictivo que pueda clasificar correctamente a los clientes.

Datos proporcionados:

Número de quejas	Porcentaje de uso (%)	Propenso a abandonar
0	100	No
2	30	Si
1	80	No
3	20	Si
0	90	No
2	50	Si
1	70	No
4	10	Si

a) Qué modelo usted sugiere que pueda predecir si un cliente va a abandonar el servicio o no, justifique su respuesta.

b) Según los datos provistos, ¿cuál sería el vector de entradas y cuál el de salida?.

c) Según su modelo sugerido y los datos provistos, graficar la frontera entre las clases e indicar los parámetros (pesos) que la definen.

d) ¿En qué tipo de ambiente se desempeñaría un agente que resuelva el problema con el modelo planteado? Indicar y justificar con una de las dimensiones:

- total/parcial observable,
- determinista/estocástico,
- episódico/secuencial,
- estático/dinámico,
- discontinuo/continuo,
- individual/multiagente,
- conocido/desconocido

✓ Resolución

a) Modelo sugerido:

Regresión logística, Perceptrón simple ó SVM lineal.

Justificación:

1. Tipo de Problema: Se trata de un problema de clasificación binaria, donde se debe decidir entre dos categorías excluyentes: "Churn" (Clase 1) o "No Churn" (Clase 0).
2. Linealmente Separable: Al visualizar los datos, se observa que un simple clasificador lineal (una línea recta) puede separar perfectamente las dos clases.
3. Cualquiera de los modelos sugeridos, brindan Simplicidad y Eficiencia: Son de los modelos más simples y computacionalmente eficientes para problemas que son linealmente separables. Dado el pequeño tamaño del conjunto de datos, un modelo más complejo podría no ser necesario y correría el riesgo de sobreajuste.

b) Vector de Entradas y de Salida

Según los datos provistos, los vectores se definen de la siguiente manera:

- Vector de Entradas (X): Es una matriz donde cada fila representa un cliente y las columnas representan sus características: "Número de quejas" (x1) y "Porcentaje de uso" (x2).

vector de entrada: [numero de quejas, porcentaje uso] vector de entrada: [0, 100]

vector de entrada aumentada: [0, 100, 1] (bias al final) ó [1, 0, 100] (bias al comienzo)

completar todos...

$X = [0, 100, 2, 30, 1, 80, 3, 20, 0, 90, 2, 50, 1, 70, 4, 10]$

$X' = [0, 100, 1, 2, 30, 1, 1, 80, 1, 3, 20, 1, 0, 90, 1, 2, 50, 1, 1, 70, 1, 4, 10, 1]$

- Vector de Salida (Y): Es un vector que contiene la clase a la que pertenece cada cliente. Asignamos 1 para "Si" (Churn) y 0 para "No" (No Churn).

$Y = [0, 1, 0, 1, 0, 1, 0, 1]$

c) graficar la frontera entre las clases e indicar los parámetros (pesos) que la definen

El objetivo de este punto es encontrar y graficar una línea recta (frontera de decisión) que separe a los clientes entre las dos clases ("Churn" y "No Churn") y especificar los parámetros o pesos que definen esa línea.

Para lograr esto, seguiremos estos pasos:

1. Visualizar los Datos: Primero, graficamos los datos proporcionados en un plano 2D.

El eje X representará el "Número de quejas" (x_1) y el eje Y representará el "Porcentaje de uso" (x_2).

- Clase 0 (No Churn): (0, 100), (1, 80), (0, 90), (1, 70).

- Clase 1 (Churn): (2, 30), (3, 20), (2, 50), (4, 10).

Al observar el gráfico, podemos notar que los grupos están claramente separados. Todos los puntos de la clase "No Churn" se encuentran a la izquierda y los de la clase "Churn" a la derecha.

2. Determinar la Frontera de Decisión

Un modelo de clasificador lineal, como el Perceptrón, crea una frontera que es una línea recta para separar las clases. La ecuación general de una recta en este plano es:

$$w_1x_1 + w_2x_2 + b = 0$$

donde:

- w_1 y w_2 son los pesos (parámetros) que indican la inclinación de la línea.
- x_1 y x_2 son las variables de entrada (quejas y uso).
- b es el sesgo (bias), que desplaza la línea.

Al colocar los puntos en el grafico se puede observar que los datos se pueden separar:

1. Con una línea vertical. Todos los clientes que no abandonan el servicio tienen 1 queja o menos, mientras que los que sí lo hacen tienen 2 o más. Por lo tanto, podemos trazar una línea vertical justo en medio de estos dos grupos, por ejemplo, en $x_1 = 1.5$.

La ecuación de esta línea es: $x_1 = 1.5$

Para que coincida con la forma general, la reescribimos como: $1 \cdot x_1 + 0 \cdot x_2 - 1.5 = 0$

2. Con una línea en diagonal. Como los parámetros no se pueden calcular a mano, sino que son el resultado del entrenamiento del modelo con los datos, se puede usar algunos valores estimados, que logren visualizar la recta que separa las dos clases.

3. Indicar los Parámetros (Pesos)

Si usamos el primer gráfico, más intuitivo y simple. Al comparar la ecuación de nuestra línea con la forma general $w_1x_1 + w_2x_2 + b = 0$, podemos identificar directamente los parámetros que la definen:

- Peso para "Número de quejas" (w_1): 1
- Peso para "Porcentaje de uso" (w_2): 0 (Esto tiene sentido, ya que la clasificación en este modelo simple no depende del porcentaje de uso, solo del número de quejas).
- Sesgo (b): -1.5

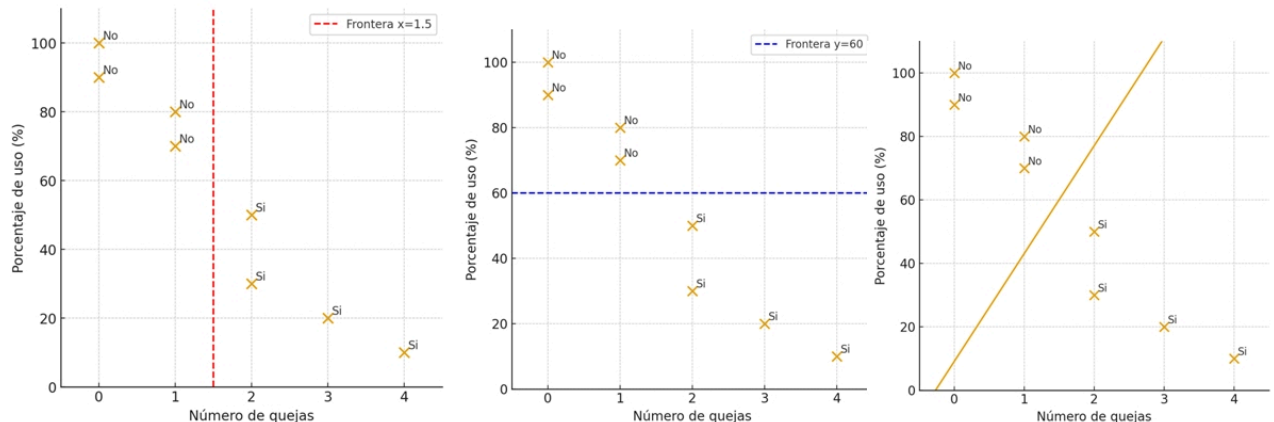
Estos parámetros definen un clasificador que predice "Churn" si $x_1 - 1.5 \geq 0$ y "No Churn" si $x_1 - 1.5 < 0$.

4. Graficar la Frontera entre las Clases

Si usamos el primer gráfico, dibujamos la línea $x_1 = 1.5$ en la gráfica junto con los puntos de datos para visualizar cómo separa las dos clases.

Como se puede ver, la línea vertical divide el espacio en dos regiones, clasificando correctamente a todos los clientes del conjunto de datos proporcionado.

Se observa que los datos se pueden separar perfectamente con una línea recta. Todos los clientes que no abandonan el servicio tienen 1 queja o menos, mientras que los que sí lo hacen tienen 2 o más.



d) Tipo de ambiente en que se desempeñaría un agente que use el modelo planteado — indicar y justificar cada dimensión

1. Totalmente observable / Parcialmente observable:

Parcialmente observable. Justificación: el modelo usa sólo dos atributos (quejas y % uso). En la práctica existen otros factores (satisfacción, precio, competencia, soporte) que no están observados; por tanto el agente tiene observación parcial del estado real del cliente.

2. Determinístico / Estocástico:

Estocástico. Justificación: la relación entre características observadas y abandono no es determinística: dos clientes con las mismas características pueden comportarse distinto. Por lo tanto, el resultado no es determinista.

3. Episódico / Secuencial:

Episódico (en la forma planteada). Justificación: cada predicción de churn aquí se hace por cliente con sus características actuales (instancia independiente). Cada predicción de cliente es un evento independiente. Clasificar a un cliente no influye en la clasificación del siguiente. El agente no necesita recordar acciones pasadas. Si se considerara evolución temporal (monitoreo a lo largo de meses) sería secuencial, pero el enunciado plantea una clasificación de instances independientes.

4. Estático / Dinámico:

Estático (en la forma planteada). Justificación: el modelo decide con las características actuales; el entorno no cambia durante la decisión. Si el entorno temporalmente cambia tras la acción (p. ej. contacto comercial), sería dinámico.

5. Discreto / Continuo:

Discreto (salida). Las acciones del agente (clasificar como Churn o No Churn) son discretas. Las entradas son una mezcla (quejas es discreto, uso es continuo pero se presenta como entero), pero el espacio de estados y acciones es fundamentalmente discreto. Justificación: la acción de clasificar es discreta (Si/No). Las características incluyen continuas (% uso) y discretas (nº quejas).

6. Individual / Multi-agente:

Individual. Justificación: el agente examina cada cliente individualmente y decide su clasificación. El agente clasificador opera de forma individual y no interactúa ni compete con otros agentes.

7. Conocido / Desconocido (sobre el entorno):

Desconocido. Justificación: El agente no conoce las reglas exactas que determinan si un cliente abandonará el servicio. Debe aprender estas reglas a partir de los datos (por eso aprendemos un modelo).

✓ Ejercicio 2

Usando el enunciado del ejercicio 1 y suponiendo que, luego de aplicar algún modelo con 200 datos de prueba, se obtienen los siguientes resultados: $TP(VP) = 70$ (verdaderos positivos), $TN(VN) = 40$ (verdaderos negativos), $FP(FP) = 30$ (falsos positivos), $FN(FN) = 60$ (falsos negativos)

Se pide:

- a) Graficar e interpretar la matriz de confusión. (5%)
- b) Si el clasificador fuera perfecto y teniendo en cuenta el punto anterior (matriz de confusión), ¿Cómo quedaría la matriz de confusión?. Escribir los valores.
- c) Según los valores anteriores, interpretar las métricas de Precisión, Exactitud (Accuracy), Sensibilidad (Recall) y Especificidad (Specificity), para el modelo dado.

✓ Resolución

a) Matriz de confusión

	Predicción: Positivo	Predicción: Negativo
Real: Positivo	TP (VP) = 70	FN (FN) = 60
Real: Negativo	FP (FP) = 30	TN (VN) = 40

Interpretación:

- **Verdaderos Positivos (TP/VP):** 70 casos donde el modelo predijo positivo y el valor real también era positivo.
- **Verdaderos Negativos (TN/VN):** 40 casos donde el modelo predijo negativo y el valor real también era negativo.
- **Falsos Positivos (FP):** 30 casos donde el modelo predijo positivo, pero el valor real era negativo (Error de Tipo I).
- **Falsos Negativos (FN):** 60 casos donde el modelo predijo negativo, pero el valor real era positivo (Error de Tipo II).

b) Matriz de Confusión para un Clasificador Perfecto

Un clasificador perfecto no comete errores, por lo tanto, los Falsos Positivos (FP) y Falsos Negativos (FN) serían cero.

- Total de clientes "No Churn" reales: $TN + FP = 40 + 30 = 70$.
- Total de clientes "Churn" reales: $TP + FN = 70 + 60 = 130$.

La matriz perfecta quedaría así:

	Predicción: Positivo	Predicción: Negativo
Real: Positivo	TP (VP) = 70	FN (FN) = 0
Real: Negativo	FP (FP) = 0	TN (VN) = 130

c) Usando los valores de la matriz de confusión:

$$TP = 70$$

$$TN = 40$$

FP = 30

FN = 60

Precisión (Precision):

- Formula: $VP/(VP + FP)$
- Cálculo: $70/(70 + 30) = 70/100 = 0.7$
- Interpretación: De todas las instancias predichas como positivas, el 70% fueron realmente positivas.

Exactitud (Accuracy):

- Formula: $(VP + VN)/(VP + FP + FN + VN)$
- Cálculo: $(70 + 40)/(70 + 30 + 60 + 40) = 110/200 = 0.55$
- Interpretación: El modelo clasificó correctamente el 55% del total de instancias.

Sensibilidad (Recall):

- Formula: $VP/(VP + FN)$
- Cálculo: $70/(70 + 60) = 70/130 \approx 0.538$
- Interpretación: De todas las instancias positivas reales, el modelo identificó correctamente aproximadamente el 53.8%.

Especificidad (Specificity):

- Formula: $VN/(VN + FP)$
- Cálculo: $40/(40 + 30) = 40/70 \approx 0.571$
- Interpretación: De todas las instancias negativas reales, el modelo identificó correctamente aproximadamente el 57.1%.

Comentario global sobre las métricas:

- Precisión relativamente buena (70%) —cuando predice abandono tiene probabilidad razonable de estar en lo cierto— pero recall es bajo-moderado ($\approx 54\%$), es decir, el sistema deja escapar muchos abandonos reales (riesgo comercial).

✓ Ejercicio 3

Usando los datos del ejercicio 1, construir el Vector de Datos de Entrenamiento y Salida

- ¿Cómo está constituido el vector de datos de entrenamiento?
- ¿Cuántas dimensiones tiene?
- ¿Qué tipos de atributos?

Resolución

Para construir los vectores de entrada y salida para entrenar una red neuronal con los datos proporcionados:

Vector de Entrada (Características/Features):

El vector de entrada estará compuesto por las características de cada cliente que utilizaremos para predecir si abandonará o no. En este caso, las características son:

- Número de quejas
- Porcentaje de uso (%)

Cada fila de los datos originales se convierte en un vector de entrada.

Vector de Salida (Etiqueta/Label):

El vector de salida representa la clase a la que pertenece cada cliente, es decir, si es propenso a abandonar el servicio (Churn) o no (No churn). Necesitamos representar estas categorías de forma numérica para la red neuronal. Una forma común es usar:

- **0** para "No churn"
- **1** para "Churn"

Aquí está cómo se verían los vectores de entrada y salida para cada ejemplo del conjunto de datos provisto:

Número de quejas	Porcentaje de uso (%)	Propenso a abandonar	Vector de Entrada	Vector de Salida
0	100	No	[0, 100]	[0]
2	30	Si	[2, 30]	[1]
1	80	No	[1, 80]	[0]
3	20	Si	[3, 20]	[1]
0	90	No	[0, 90]	[0]
2	50	Si	[2, 50]	[1]
1	70	No	[1, 70]	[0]
4	10	Si	[4, 10]	[1]

- **¿Cuántas dimensiones tiene el vector de entrada?** El vector de entrada tiene **2 dimensiones**, una por cada característica ("Número de quejas" y "Porcentaje de uso").
- **¿Qué tipos de atributos tiene el vector de entrada?** Ambos atributos ("Número de quejas" y "Porcentaje de uso") son de tipo **numérico**. "Número de quejas" es una variable discreta (conteo), mientras que "Porcentaje de uso" es una variable continua (un porcentaje). Para la mayoría de los modelos de redes neuronales, ambos pueden ser tratados como valores numéricos de punto flotante.

Ejercicio 4

Usando los datos del ejercicio 1, ¿Cómo serían dos pasos de entrenamiento utilizando un perceptrón simple, con un bias de 1 y una tasa de aprendizaje (α) de 0.3?

✓ Ejemplo de Entrenamiento con un Perceptrón Simple

Vamos a demostrar dos pasos de entrenamiento utilizando un perceptrón simple con los siguientes parámetros:

- **Bias inicial:** 1
- **Tasa de aprendizaje (α):** 0.3

Usaremos los dos primeros ejemplos del conjunto de datos para la demostración.

Datos de entrenamiento:

Número de quejas	Porcentaje de uso (%)	Propenso a abandonar	Vector de Entrada (x)	Vector de Salida (y)
0	100	No	[0, 100]	0
2	30	Si	[2, 30]	1

Inicialización de pesos:

Comenzaremos con pesos iniciales aleatorios (o podríamos inicializarlos a cero por simplicidad en este ejemplo, pero usaremos valores pequeños aleatorios para una demostración más realista).

$$w = [w_1, w_2]$$

$$b = 1$$

Pesos iniciales (aleatorios pequeños): $w = [0.1, -0.05]$

Consideramos el vector ampliado donde el bias b es representado por w_3

Pesos iniciales (aleatorios pequeños): $w = [0.1, -0.05, 1]$

Paso 1: Entrenamiento con el primer ejemplo

Entrada (x): [0, 100, 1]

Salida deseada (y): 0

1. Calcular la entrada neta (z):

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3$$

$$z = (0.1 \cdot 0) + (-0.05 \cdot 100) + 1$$

$$z = 0 - 5 + 1 = -4$$

2. Aplicar la función de activación (escalón):

Para un perceptrón simple, usamos una función escalón. Si $z \geq 0$, la salida es 1; si $z < 0$, la salida es 0.

Salida predicha ($\hat{y}$): Como $z = -4 < 0$, $\hat{y} = 0$.

3. Calcular el error:

$$\text{Error} = y - \hat{y}$$

$$\text{Error} = 0 - 0 = 0$$

4. Actualizar pesos y bias:

Como el error es 0, no hay actualización de pesos ni bias en este paso. Los pesos y bias se mantienen igual.

$$w_{\text{nuevo}} = w_{\text{viejo}}$$

Pesos después del paso 1 (incluyendo w_3): $w = [0.1, -0.05, 1]$

Paso 2: Entrenamiento con el segundo ejemplo

Entrada (x): $[2, 30, 1]$

Salida deseada (y): 1

1. Calcular la entrada neta (z):

Usamos los pesos y bias actualizados (que en este caso no cambiaron).

$$z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3$$

$$z = (0.1 \cdot 2) + (-0.05 \cdot 30) + 1$$

$$z = 0.2 - 1.5 + 1 = -0.3$$

2. Aplicar la función de activación (escalón):

Salida predicha ($\hat{y}$): Como $z = -0.3 < 0$, $\hat{y} = 0$.

3. Calcular el error:

$$\text{Error} = y - \hat{y}$$

$$\text{Error} = 1 - 0 = 1$$

4. Actualizar pesos y bias:

Como el error es 1, actualizamos los pesos y el bias usando la regla de actualización del perceptrón:

$$w_{i,\text{nuevo}} = w_{i,\text{viejo}} + \alpha \cdot \text{Error} \cdot x_i$$

Actualización del peso w_1 :

$$w_{1,\text{nuevo}} = w_{1,\text{viejo}} + \alpha \cdot \text{Error} \cdot x_1$$

$$w_{1,\text{nuevo}} = 0.1 + 0.3 \cdot 1 \cdot 2$$

$$w_{1,\text{nuevo}} = 0.1 + 0.6 = 0.7$$

Actualización del peso w_2 :

$$w_{2,\text{nuevo}} = w_{2,\text{viejo}} + \alpha \cdot \text{Error} \cdot x_2$$

$$w_{2,\text{nuevo}} = -0.05 + 0.3 \cdot 1 \cdot 30$$

$$w_{2,\text{nuevo}} = -0.05 + 9 = 8.95$$

Actualización del peso w_3 (bias b):

$$w_{3,\text{nuevo}} = w_{3,\text{viejo}} + \alpha \cdot \text{Error}$$

$$w_{3,\text{nuevo}} = 1 + 0.3 \cdot 1$$

$$w_{3,\text{nuevo}} = 1 + 0.3 = 1.3$$

Resultados después de dos pasos de entrenamiento:

Pesos: $w = [0.7, 8.95, 1.3]$

Este es un ejemplo simple de cómo el perceptrón ajusta sus pesos y bias en cada paso de entrenamiento basado en el error para aprender a clasificar los datos.

Con más iteraciones y datos, el perceptrón (si el problema es linealmente separable) convergerá a un conjunto de pesos y bias que definen la frontera de decisión.

✓ Ejercicio 5

Calcular el resultado de una capa de **maxpooling** con ventana de tamaño **2x2**, paso **(stride) 1x1** y **sin padding** de la entrada.

10	5	2	5	8
4	1	4	7	2
15	1	6	4	12
6	4	8	4	10
12	4	8	10	8

10, 5, 7, 8

15, 6, 7, 12

15, 8, 8, 12

12, 8, 10, 10

✓ Ejercicio 6

Dada la siguiente matriz m (3x3) realizar una convolución aplicando el kernel k (3x3), y luego, sobre la matriz resultante, aplicar un MAX pooling, de tamaño 2x2 con paso = 1

$$m = \begin{bmatrix} 1 & 5 & 10 \\ 2 & 10 & 10 \\ 3 & 7 & 5 \end{bmatrix}$$

$$k = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

NOTA: Luego de la convolución la matriz resultante debe tener el mismo tamaño de la matriz m

Resolución convolución

1. Primero aplicamos padding para considerar todos las entradas: se agregan 0 alrededor de la matriz de entrada:

$$m = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 5 & 10 & 0 \\ 0 & 2 & 10 & 10 & 0 \\ 0 & 3 & 7 & 5 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

2. Operamos sobre el primer elemento de la matriz de entrada usando el kernel:

$$conv = \sum_{i=1}^n \sum_{j=1}^m W_{ij} M_{ij}$$

donde

$$z = 1a + 2b + 3c + 4d + 5e + 6f + 7g + 8h + 9p$$

$$m = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 5 \\ 0 & 2 & 10 \end{bmatrix}$$

*

$$k = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$$

$$r_1 = (0 * 1 + 0 * 0 + 0 * (-1)) + (0 * 0 + 1 * 0 + 5 * 0) + (0 * (-1) + 2 * 0 + 10 * 0) \\ r_1 = (0) + (0) + (10) \quad r_1 = 10$$

3. Operamos sobre el segundo elemento de la matriz de entrada usando el kernel:

$$m = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 5 & 10 \\ 2 & 10 & 10 \end{bmatrix}$$

$$r_2 = (0 * 1 + 0 * 0 + 0 * (-1)) + (1 * 0 + 5 * 0 + 10 * 0) + (2 * (-1) + 10 * 0 + 10 * 0) + 1$$

$$r_2 = (0) + (0) + (8) \quad r_2 = 8$$

4. Operamos sobre el tercer elemento de la matriz de entrada usando el kernel:

$$m = \begin{bmatrix} 0 & 0 & 0 \\ 5 & 10 & 0 \\ 10 & 10 & 0 \end{bmatrix}$$

$$r_3 = (0 * 1 + 0 * 0 + 0 * (-1)) + (5 * 0 + 10 * 0 + 0 * 0) + (10 * (-1) + 10 * 0 + 10 * 0) + 1$$

$$r_3 = (0) + (0) + (-10) \quad r_3 = -10$$

4. continuamos con todos hasta llegar al resultado

$$r = \begin{bmatrix} 10 & 8 & -10 \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

Aplicar Max pooling

$$r = \begin{bmatrix} 10 & 8 & -10 \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}$$

$$p = \begin{bmatrix} ? & ? \\ ? & ? \end{bmatrix}$$

✓ Ejercicio 7

• ¿Cómo se calculan los pesos en Backpropagation?

En Backpropagation, los pesos de una red neuronal se ajustan iterativamente utilizando el algoritmo de descenso de gradiente. El proceso general es el siguiente:

1. **Paso hacia adelante (Forward Pass):** La entrada se propaga a través de la red, calculando las salidas de cada neurona en cada capa.
2. **Cálculo del Error:** Se compara la salida final de la red con la salida deseada (etiqueta real) y se calcula un error, a menudo utilizando una función de pérdida (por ejemplo, error cuadrático medio o entropía cruzada).
3. **Paso hacia atrás (Backward Pass):** El error se propaga hacia atrás a través de la red, desde la capa de salida hasta la capa de entrada. Durante este proceso, se calcula el gradiente del error con respecto a cada peso de la red. El gradiente indica la dirección y magnitud del cambio en el error para un pequeño cambio en el peso.
4. **Actualización de Pesos:** Los pesos se actualizan en la dirección opuesta al gradiente para minimizar el error. La fórmula de actualización de pesos es

típicamente:

$$w_{\text{nuevo}} = w_{\text{viejo}} - \alpha \cdot \frac{\partial E}{\partial w}$$

Donde:

- w_{nuevo} es el nuevo valor del peso.
- w_{viejo} es el valor actual del peso.
- α es la tasa de aprendizaje, un hiperparámetro que controla el tamaño de los pasos de actualización.
- $\frac{\partial E}{\partial w}$ es el gradiente del error (E) con respecto al peso (w).

Este proceso se repite durante múltiples épocas (pasadas completas a través del conjunto de datos de entrenamiento) hasta que el error se minimiza o se alcanza un criterio de convergencia.

<https://www.youtube.com/watch?v=tLY-gNoGEPs>

✓ Ejercicio 8

¿Qué pasa si inicialmente los bias son todos iguales a 1?

Inicialización de Bias a 1 en Redes Neuronales: Problemas y Consecuencias

Principales Problemas

1. Ruptura de Simetría Insuficiente

- Aunque es mejor que inicializar a 0, sigue siendo problemático
- Todas las neuronas en la misma capa comienzan con el mismo sesgo
- Las neuronas pueden aprender representaciones muy similares en las primeras iteraciones
- Reduce la diversidad en el aprendizaje de características

2. Gradientes Desbalanceados

- Todas las neuronas comienzan con la misma predisposición
- Patrones de gradientes similares durante backpropagation
- Limita la capacidad de la red para aprender características diversas

3. Problemas Específicos por Función de Activación

ReLU:

```
# ReLU( $w x + 1$ ) =  $\max(0, w x + 1)$   
# Con bias = 1, la neurona casi siempre estará activa inicialmente  
# Puede no ser óptimo para todas las neuronas
```

Leaky ReLU:

```
# Similar a ReLU, predisposición hacia activación  
# Puede reducir el beneficio de la "muerte" selectiva de neuronas
```

✓ Posibles Beneficios (Limitados)

1. Previene Neuronas "Muertas" con ReLU

- Un bias positivo puede evitar que las neuronas se "mueran" (salida siempre 0)
- Especialmente útil en las primeras iteraciones del entrenamiento

2. Rompe Simetría Básica

- Es mejor que inicializar todos los bias a 0
- Evita que todas las neuronas sean completamente idénticas

Mejores Alternativas de Inicialización

1. Inicialización a Cero (Recomendado)

```
import torch.nn as nn  
  
# PyTorch - inicialización por defecto  
linear_layer = nn.Linear(input_size, output_size)  
# Los bias se inicializan automáticamente cerca de 0  
  
# Inicialización manual a 0  
nn.init.zeros_(linear_layer.bias)
```

2. Inicialización Aleatoria Pequeña

```
import torch.nn.init as init  
  
# Distribución normal con media 0 y desviación estándar pequeña  
init.normal_(linear_layer.bias, mean=0, std=0.01)  
  
# Distribución uniforme pequeña  
init.uniform_(linear_layer.bias, -0.01, 0.01)
```

3. Inicialización Específica por Función de Activación

```
# Para ReLU - a menudo se usa 0
init.zeros_(relu_layer.bias)

# Para funciones sigmoidal - valores muy pequeños
init.normal_(sigmoid_layer.bias, mean=0, std=0.01)
```

4. Inicialización Adaptativa

```
def custom_bias_init(layer, activation_type):
    if activation_type == 'relu':
        init.zeros_(layer.bias)
    elif activation_type in ['sigmoid', 'tanh']:
        init.normal_(layer.bias, mean=0, std=0.01)
    elif activation_type == 'leaky_relu':
        init.uniform_(layer.bias, -0.01, 0.01)
```



Comparación de Estrategias

Estrategia	Ventajas	Desventajas	Casos de Us
Bias = 1	Previene neuronas muertas	Saturación, gradientes similares	✗ No recomend
Bias = 0	Simple, funciona bien	Posibles neuronas muertas con ReLU	✓ Uso general
Aleatorio pequeño	Diversidad inicial	Complejidad adicional	✓ Casos especí
Específico por activación	Optimizado para cada función	Requiere más conocimiento	✓ Redes compl



Conclusión

Inicializar todos los bias a 1 generalmente tiene más perjuicios que beneficios:

- ✗ **Problemas principales:** Saturación de activaciones, gradientes similares, aprendizaje lento
- ✗ **Impacto en backpropagation:** Limita la diversidad en el aprendizaje de características
- ✓ **Alternativas recomendadas:** Inicialización a 0 o valores aleatorios muy pequeños
- 🔧 **Mejor práctica:** Usar inicialización específica según la función de activación y arquitectura de la red

La inicialización adecuada de parámetros es crucial para el entrenamiento eficiente de redes neuronales profundas.

Ejercicio 9

Tipos de problemas a resolver por redes neuronales

1. ¿Cuáles son los dos grandes grupos de problemas que se pueden plantear mediante el uso de redes neuronales artificiales?
2. ¿Qué relación tienen las funciones de activación con respecto al tipo de problemas a resolver?
3. ¿Es posible mezclar, en una red neuronal profunda, capas con distintos tipos de funciones de activación? ¿Qué se puede lograr con eso?

✓ Grupos de Problemas según Separabilidad

Problemas Linealmente Separables

Problemas donde los datos pueden ser separados por una función lineal (hiperplano).

Características

Los datos de diferentes clases pueden separarse con una línea recta (2D), plano (3D) o hiperplano (nD) Una sola capa neuronal (perceptrón) puede resolver el problema No requieren funciones de activación no lineales en capas ocultas Convergencia garantizada con algoritmos apropiados

Problemas No Linealmente Separables

Problemas donde no existe ningún hiperplano que pueda separar correctamente los datos.

Características

Los datos de diferentes clases están entremezclados de forma compleja Requieren múltiples capas con funciones de activación no lineales Mayor complejidad computacional Requieren más datos y tiempo de entrenamiento

Funciones de Activación según Separabilidad

Activación	Ventajas para No Linealidad	Desventajas
ReLU	Simple, evita desvanecimiento, crea regiones lineales por partes	Neuronas muertas
Sigmoid	Suave, diferenciable en todo punto	Desvanecimiento de gradiente
Tanh	Centrada en cero, rango [-1,1]	Desvanecimiento en redes profundas
Leaky ReLU	Evita neuronas muertas	Hiperparámetro α
Swish/SiLU	Auto-regulada, muy suave	Computacionalmente más costosa

Estrategia de Diseño

Tipo de Problema	Arquitectura Recomendada	Activaciones Clave	Complejidad
Linealmente Separable	1-2 capas	ReLU simple	Baja
Parcialmente Separable	2-3 capas	ReLU + Tanh	Media
No Linealmente Separable	3+ capas	ReLU + ELU + Swish	Alta
Altamente Complejo	Redes profundas	Activaciones múltiples	Muy Alta

Principios de Diseño

- Principio de Parsimonia: Usar la arquitectura más simple que resuelva el problema
- Evaluación Empírica: Probar primero con modelos simples
- Escalabilidad: Diseñar arquitecturas que puedan crecer en complejidad
- Especialización: Usar activaciones específicas para cada etapa del procesamiento

Implicaciones Teóricas

Teorema de Aproximación Universal

Cualquier función continua puede ser aproximada por una red con al menos una capa oculta y activación no lineal.

Para problemas linealmente separables, esto es "overkill", pero garantiza robustez.

Para problemas no linealmente separables, es absolutamente necesario

Ejercicio 10

Redes Neuronales Convolucionales (CNN): Guía Concisa

Características Fundamentales de las CNN

Característica	Descripción	Beneficio
Conectividad Local	Cada neurona conecta solo a una región pequeña (ej: 3×3)	Menos parámetros, es
Compartición de Parámetros	Mismo kernel para toda la imagen	Invarianza traslacional
Jerarquía de Características	Capas tempranas → básicas, Capas profundas → complejas	Construcción progresiva
Estructura Espacial	Preserva relaciones geométricas	Crucial para reconocer

Flujo de Abstracción

- **Capa 1:** Bordes, líneas, colores
- **Capa 2:** Esquinas, curvas, texturas
- **Capa 3:** Formas, patrones complejos

- **Capa 4+:** Objetos parciales, características de alto nivel

Kernels (Filtros)

¿Qué es un Kernel?

- **Matriz pequeña de pesos** (típicamente 3×3, 5×5)
- Se **desliza** sobre la imagen
- **Detecta características específicas** mediante convolución
- Los pesos se **aprenden automáticamente** durante entrenamiento

Tipos de Kernels por Función

Tipo	Propósito	Ejemplo de Uso
Detección de Bordes	Identifica transiciones de intensidad	Bordes horizontales, verticales, diagonales
Suavizado	Reduce ruido y detalles	Filtros Gaussianos, promedio
Enfoque	Realza detalles y contornos	Sharpening, realce de características
Especializados	Tareas específicas	Detección de esquinas, texturas

Parámetros Clave

- **Stride:** Pasos del desplazamiento (1, 2, 3...)
- **Padding:** Relleno alrededor de la imagen
- **Cantidad:** Múltiples kernels por capa (32, 64, 128...)

Convolución vs Pooling

Aspecto	Convolución	Pooling
Función	Extrae características	Reduce dimensionalidad
Parámetros	✅ Aprendibles	❌ Fijos
Información	Transforma datos	Submuestra datos
Resolución	Puede mantener	Siempre reduce
Canales	Puede aumentar	Se mantiene
Propósito	Detección de patrones	Invarianza + eficiencia

Tipos de Pooling

Tipo	Operación	Cuándo Usar
Max Pooling	Valor máximo de la ventana	Características más prominentes
Average Pooling	Promedio de la ventana	Representación suavizada
Global Pooling	Un valor por canal	Clasificación final

Usos de Cada Tipo de Capa

Capas Convolucionales - Para:

-  **Extraer características** específicas de la imagen
-  **Detectar patrones** (bordes, texturas, formas)
-  **Preservar información espacial** y relaciones geométricas
-  **Transformar representaciones** de píxeles a características
-  **Crear jerarquías** de características simples → complejas

Capas de Pooling - Para:

-  **Reducir dimensiones** espaciales (eficiencia computacional)
-  **Crear invarianza** a pequeñas traslaciones
-  **Controlar overfitting** (menos parámetros en capas siguientes)
-  **Extraer características dominantes** (eliminar detalles menores)
-  **Aumentar campo receptivo** (cada neurona "ve" más región)



Arquitectura Típica CNN

Patrón Clásico

Input → [Conv → Activación → Pool] × N → Flatten → Dense → Output

Ejemplo de Flujo Dimensional

Etapas	Entrada	Salida	Propósito
Input	224×224×3	-	Imagen RGB original
Conv1	224×224×3	224×224×32	Detecta bordes básicos
Pool1	224×224×32	112×112×32	Reduce dimensiones
Conv2	112×112×32	112×112×64	Características intermedias
Pool2	112×112×64	56×56×64	Mayor abstracción
Conv3	56×56×64	56×56×128	Características complejas
Pool3	56×56×128	28×28×128	Preparación final
Dense	28×28×128	10 classes	Clasificación

Beneficios del Patrón Conv-Pool

- **Extracción + Reducción:** Complementariedad perfecta
- **Eficiencia progresiva:** Menos datos en cada etapa
- **Jerarquía natural:** Simple → Complejo automáticamente
- **Invarianza creciente:** Mayor robustez a variaciones



Comparación: CNN vs Redes Tradicionales

Característica	CNN	Red Densa
Parámetros	Compartidos (pocos)	Independientes (muchos)
Conectividad	Local	Global

Característica	CNN	Red Densa
Estructura espacial	Preservada	Perdida
Invarianza traslacional	✓ Inherente	✗ No
Eficiencia con imágenes	✓ Alta	✗ Muy baja
Especialización	✓ Automática	✗ Manual

Criterios de Decisión

Usar CNN Cuando:

- Datos tienen **estructura espacial** (imágenes, audio, series temporales)
- Necesitas **detectar patrones** independientes de posición
- Requieres **eficiencia paramétrica**
- Buscas **invarianza traslacional**

Usar Convolucionales Cuando:

- Extraer **nuevas características**
- Detectar **patrones específicos**
- Preservar **relaciones espaciales**
- Transformar **representaciones**

Usar Pooling Cuando:

- Reducir **costo computacional**
- Aumentar **invarianza a traslaciones**
- Controlar **overfitting**
- Enfocar en **características dominantes**

Conclusiones Clave

Por qué CNN Revolucionaron la Visión por Computadora:

1. **Arquitectura especializada** para datos espaciales
2. **Aprendizaje automático** de detectores de características
3. **Eficiencia paramétrica** masiva vs redes densas
4. **Invarianza traslacional** incorporada
5. **Jerarquía natural** de abstracciones

Principio Fundamental:

- **Convolución** = Extracción y detección de características
- **Pooling** = Reducción y creación de invarianza
- **Combinación** = Sistema eficiente para reconocimiento visual

Impacto:

- Hicieron posible el **reconocimiento automático** a gran escala
- Base de aplicaciones modernas: **reconocimiento facial, coches autónomos, diagnóstico médico**
- Inspiraron arquitecturas para **otros dominios** (NLP, audio)